

Parallel Tool Calls and the Step Budget, Worked Out by Hand

Two knobs, two different axes: `asyncio.gather` shrinks the latency *inside* one step; `max_steps` bounds cost and time *across* all steps.

What this document does. It takes the one tiny scenario from Module 0.2 — the weather agent that, in a single turn, fires `get_weather(Bhopal)`, `get_weather(Delhi)`, and `get_weather(Mumbai)` at once — and counts the milliseconds. We compute the sequential time, the parallel (`asyncio.gather`) time, and the speedup, first for three equal calls and then for three unequal ones, by hand. Then we turn to the loop’s other knob, `max_steps`, and show how a single integer M converts an unbounded `while` loop into a hard linear ceiling on calls, cost, and latency. Every number below is reproduced by the small Python program in Appendix A. The module quotes “3 sequential calls @ 400ms = 1200ms, parallel = ~400ms, 3× faster”; we rebuild that figure from first principles, then show what changes when the calls are *not* equal.

Where this sits. This is **Topic 1 of Module 0.2** (Function Calling From Scratch). Its companions: Module 0.0 Topic 1 (the cost of the ReAct loop — the *critical-path* idea reappears here as the slowest call) and Module 0.1 (context-window pressure — the other thing the step budget protects).

Concepts covered, in order: fan-out within one step · sequential time $T_{\text{seq}} = \sum L_i$ · parallel time $T_{\text{par}} = \max L_i$ · speedup $= \sum L_i / \max L_i$, bounded by n · the slowest call dominates (critical path) · Amdahl: the serial *decide* is untouched · the step budget M as a linear hard bound · worst-case cost $\leq M c$ and latency $\leq M L$ · two knobs, two axes.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one turn, three tool calls, small enough to time by hand

A modern model can emit several tool calls in a single assistant turn. The weather agent from the module does exactly this: asked to compare three cities, it returns three independent `get_weather` calls *at once*. The runtime now has a choice. It can run them one after another (sequential), or it can launch all three together and wait for them to finish (parallel, via `asyncio.gather`). The tool calls do not depend on each other, so both produce the same observations — but they take very different amounts of wall-clock time.

The quantities. We give every tool call a latency in milliseconds. These are illustrative but realistic for a network call, and chosen round so you can follow the sums on paper. The mechanics are identical at any size — only the constants change.

Quantity	Symbol	Value here
Number of parallel tool calls in the turn	n	3
Latency of call i (network round-trip)	L_i	see below
equal case (module’s example)		400, 400, 400 ms
unequal case		400, 250, 700 ms
Serial “decide” time (the LLM turn itself)	D	200 ms
Per-step latency ceiling (step budget)	L	1,000 ms
Per-call cost ceiling (step budget)	c	\$0.02
Step budget	M	5, 10, 20

Notation. T_{seq} is the wall-clock time to run all n calls one at a time; T_{par} is the wall-clock time to run them concurrently. The *speedup* is $T_{\text{seq}}/T_{\text{par}}$. The “decide” time D is the LLM call that produced the tool calls; it is serial and happens before any tool runs.

Intuition

Sequential tool calls are like asking three weather desks one at a time and waiting at each window before walking to the next: you stand in every queue back-to-back. `asyncio.gather` is sending three runners to the three windows simultaneously — you are done when the *slowest* runner returns. Nobody waits for anybody else. The only thing you cannot parallelise away is the time it took you to *decide* to send three runners in the first place.

2 Step 1 — sequential vs parallel within one step

Run the n calls one after another and the times simply add up. Run them together and you wait only for the longest one to finish:

$$T_{\text{seq}} = \sum_{i=1}^n L_i, \quad T_{\text{par}} = \max_i L_i. \quad (1)$$

The speedup is their ratio:

$$\text{speedup} = \frac{T_{\text{seq}}}{T_{\text{par}}} = \frac{\sum_{i=1}^n L_i}{\max_i L_i} \quad (2)$$

Equal case (the module’s example). Three calls at 400 ms each: $T_{\text{seq}} = 400+400+400 = 1200$ ms, and $T_{\text{par}} = \max(400, 400, 400) = 400$ ms. Speedup = $1200/400 = 3.00\times$. ✓ That is exactly the module’s quoted “1200ms $\rightarrow$ $\sim$ 400ms, $3\times$ faster.”

Mini-example — this operation in isolation

Why is $3\times$ the *best* you can do with $n = 3$? Each numerator term L_i is at most $\max_i L_i$, so $\sum_i L_i \leq n \cdot \max_i L_i$, hence speedup = $\sum L_i / \max L_i \leq n$. Equality holds *only* when every L_i equals the maximum — i.e. when all calls take the same time. The instant the calls differ, the faster ones finish early and sit idle waiting for the slowest, so you recover less than the full $n\times$. Parallelism cannot beat the number of things you ran in parallel.

3 Step 2 — unequal calls: the slowest one dominates

Real tool calls are not equal. Take $L = (400, 250, 700)$ ms. Sequentially, $T_{\text{seq}} = 400 + 250 + 700 = 1350$ ms. In parallel, $T_{\text{par}} = \max(400, 250, 700) = 700$ ms — the single 700 ms call sets the floor; the 400 and 250 ms calls finish and wait. The speedup is

$$\text{speedup} = \frac{1350}{700} = 1.9286 \dots \approx 1.93 \times .$$

✓ Less than $3\times$, and less than the equal case's $3\times$, even though we ran the *same* number of calls. The 700 ms call is the **critical path**: the turn cannot finish before it does, so shaving the other two to zero would change nothing.

Case	L_1, L_2, L_3 (ms)	$T_{\text{seq}} = \sum L_i$	$T_{\text{par}} = \max L_i$	speedup
Equal	400, 400, 400	1,200	400	$3.00\times$
Unequal	400, 250, 700	1,350	700	$1.93\times$

This is the same critical-path idea from Module 0.0: a chain (or here, a fan-out) is only as fast as its slowest link. Adding more fast calls barely moves T_{par} ; adding one slow call moves it a lot.

Intuition

To make a parallel turn faster, do not optimise the average tool — optimise the *slowest* one. Speeding up a 250 ms call when a 700 ms call is in the same batch buys you exactly nothing in wall-clock time. The batch is gated by its maximum, full stop. This is why agent engineers profile for the long-pole tool, not the typical tool.

4 Step 3 — Amdahl: parallelism only helps the fan-out

The tool fan-out is not the whole step. Before any tool runs, the model had to *decide* to call them — that LLM turn took $D = 200$ ms, and it is irreducibly serial. So the honest per-step time is decide *plus* the tool part:

$$T_{\text{step}}^{\text{seq}} = D + \sum_i L_i, \quad T_{\text{step}}^{\text{par}} = D + \max_i L_i.$$

For the equal case: $T_{\text{step}}^{\text{seq}} = 200 + 1200 = 1400$ ms and $T_{\text{step}}^{\text{par}} = 200 + 400 = 600$ ms, so the *overall* speedup is $1400/600 = 2.33\times$ — not $3\times$. ✓ The 200 ms serial decide is a fixed tax that parallelism cannot touch, so it drags the realised speedup below the fan-out-only figure.

Watch out

Do not quote the fan-out speedup as if it were the step speedup. The $3\times$ from Section 2 is the speedup of the *tool portion alone*. Once you add the serial decide, the step-level speedup is $2.33\times$ (equal) or $1.72\times$ (unequal). This is Amdahl's law in miniature: a fixed serial fraction caps the benefit of parallelising the rest. If the decide step grows (longer prompt, bigger model), the parallel win shrinks — even though the tools got no slower.

5 Step 4 — the step budget: an unbounded loop made bounded

Parallelism attacks time *within* a step. The step budget attacks the total *across* steps. Recall the loop’s last line from the module: `raise RuntimeError("step budget exceeded")`. Without `max_steps`, a confused model can loop forever — calling tools, re-reading history, calling more tools, with no guaranteed stop. `max_steps = M` is a single integer that makes the worst case finite. It is a **linear** hard bound:

$$\boxed{\text{LLM calls} \leq M, \quad \text{cost} \leq M \cdot c, \quad \text{latency} \leq M \cdot L} \quad (3)$$

where c is the per-call cost ceiling and L is the per-step latency ceiling. The model might finish early (it usually does), but it *cannot* exceed these. With $c = \$0.02$ and $L = 1,000$ ms per step:

Budget M	Worst-case LLM calls	Worst-case cost Mc	Worst-case latency ML
5	5	\$0.10	5,000 ms = 5.0 s
10	10	\$0.20	10,000 ms = 10.0 s
20	20	\$0.40	20,000 ms = 20.0 s

Sample check, $M = 10$: worst cost = $10 \times \$0.02 = \0.20 ; worst latency = $10 \times 1000 = 10,000$ ms = 10 s. ✓ Doubling the budget doubles both — this is the *linear* bound, and that linearity is exactly why the budget is a safe, predictable cap. (Note the per-step *cost* can itself grow with history, as Module 0.0 shows; the budget bounds the *count* of steps, which is what stops an infinite loop. Here we hold c at a per-call ceiling to isolate the count.)

Intuition

A step budget is a circuit breaker, not a performance tuning knob. It does not make any single step faster or cheaper; it guarantees the loop *terminates* and puts a ceiling you can write on a slide: “this agent costs at most \$0.40 and takes at most 20 seconds.” Without it, the honest answer to “what’s the worst case?” is “unbounded” — which no one can ship.

6 Step 5 — two knobs, two different axes

It is worth stating plainly, because conflating them is the classic mistake:

- **Parallelism** (`asyncio.gather`) reduces *within-step* latency. It turns $\sum_i L_i$ into $\max_i L_i$ for the tool calls of *one* turn. It does nothing about how many turns there are.
- **The step budget** (`max_steps`) bounds the *across-step* total. It caps the *number* of turns at M , bounding total cost and latency. It does nothing about how fast any single turn runs.

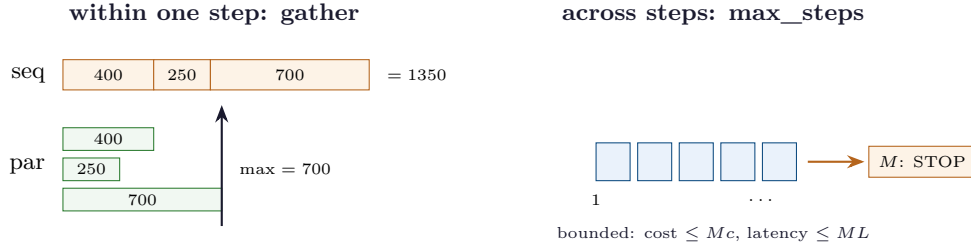
They are orthogonal. You can have a fast-but-runaway agent (great parallelism, no budget) or a bounded-but-slow one (tight budget, serial tools). A well-engineered loop uses both: `gather` to make each step cheap in time, and `max_steps` to make the whole trajectory bounded in count.

Mini-example — this operation in isolation

Put numbers on it. An agent with the equal fan-out per step runs each step in $T_{\text{step}}^{\text{par}} = 600$ ms (Section 3). Under a budget $M = 10$, the worst-case wall-clock is then $\leq 10 \times 600 = 6,000$

ms = 6 s — parallelism set the per-step figure, the budget multiplied it by the cap. Drop the parallelism and each step costs 1400 ms, so the same budget now allows up to 14,000 ms. Same budget, very different ceiling: the two knobs compose, they do not substitute.

7 The two mechanisms, drawn



Left: within a step, the three calls collapse from a 1350 ms sequence to a 700 ms stack — you wait for the tallest bar. Right: across steps, the budget M draws a wall the loop cannot cross. One shrinks a bar; the other caps a count. Different axes.

8 Cheat-sheet

Sequential time (one step)	$T_{\text{seq}} = \sum_{i=1}^n L_i$
Parallel time (gather , one step)	$T_{\text{par}} = \max_i L_i$
Speedup (tool fan-out only)	$\frac{\sum_i L_i}{\max_i L_i} \leq n$ (= n iff all L_i equal)
Critical path	the single $\max_i L_i$ call gates the turn
Step time with serial decide	$T_{\text{step}} = D + (\text{seq or par tool time})$
Step budget (linear hard bound)	$\text{calls} \leq M, \text{ cost} \leq Mc, \text{ latency} \leq ML$
Two axes	$\text{gather} \rightarrow \text{within-step latency}; M \rightarrow \text{across-step total}$

9 An honest word about these numbers

The latencies here (400, 250, 700 ms), the decide time (200 ms), and the budget ceilings ($c = \$0.02$, $L = 1,000$ ms) are illustrative round numbers, picked so the sums are followable by hand. Your real tool latencies depend on the network, the upstream API, and payload sizes; a cold cache or a rate-limited endpoint can make one call dominate far more than our 700 ms. What is *not* illustrative is the structure. Sequential time is always the sum and parallel time is always the maximum, so the speedup is always $\sum L_i / \max L_i$ and never exceeds n . A serial decide always caps the realised speedup below the fan-out figure (Amdahl). And a step budget M always converts an unbounded loop into the linear ceilings of Equation (3). Change the constants and the milliseconds and dollars move; the two shapes — “max, not sum” and “linear cap” — do not. Those are the things to carry into every later module.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the speedup figures, the Amdahl-adjusted step speedups, and the step-budget table; change the latencies, the decide time, or the ceilings to explore your own agent's numbers.

```

1  # parallel_budget.py -- reproduces every number in
2  # "Parallel Tool Calls and the Step Budget".
3
4  def seq(L): return sum(L) # T_seq = sum of latencies
5  def par(L): return max(L) # T_par = max latency (asyncio.gather)
6  def speedup(L): return seq(L) / par(L) # bounded above by n = len(L)
7
8  equal = [400, 400, 400] # ms -- the module's example
9  unequal = [400, 250, 700] # ms -- slowest call dominates
10 D = 200 # ms -- serial "decide" (the LLM turn), Amdahl tax
11 c = 0.02 # $ -- per-call cost ceiling
12 L_step = 1000 # ms -- per-step latency ceiling
13
14 # --- within-step: sequential vs parallel ---
15 for name, L in (("equal", equal), ("unequal", unequal)):
16     print(f"{name:7s} L={L} T_seq={seq(L)} T_par={par(L)} "
17           f"speedup={speedup(L):.4f}")
18
19 # bound check: speedup <= n, equality iff all equal
20 n = 3
21 print(f"equal speedup == n ? {abs(speedup(equal) - n) < 1e-9}")
22 print(f"unequal speedup < n ? {speedup(unequal) < n}")
23
24 # --- Amdahl: add the serial decide, recompute step-level speedup ---
25 for name, L in (("equal", equal), ("unequal", unequal)):
26     s = D + seq(L) # serial step time
27     p = D + par(L) # parallel step time
28     print(f"{name:7s} step_seq={s}ms step_par={p}ms "
29           f"overall_speedup={s/p:.4f}")
30
31 # --- step budget: linear hard bound on calls, cost, latency ---
32 for M in (5, 10, 20):
33     print(f"M={M:2d} calls<= {M} cost<= ${M*c:.2f} "
34           f"lat<= {M*L_step}ms = {M*L_step/1000:.1f}s")
35
36 # Expected output:
37 # equal L=[400, 400, 400] T_seq=1200 T_par=400 speedup=3.0000
38 # unequal L=[400, 250, 700] T_seq=1350 T_par=700 speedup=1.9286
39 # equal speedup == n ? True
40 # unequal speedup < n ? True
41 # equal step_seq=1400ms step_par=600ms overall_speedup=2.3333
42 # unequal step_seq=1550ms step_par=900ms overall_speedup=1.7222
43 # M= 5 calls<= 5 cost<= $0.10 lat<= 5000ms = 5.0s
44 # M=10 calls<= 10 cost<= $0.20 lat<= 10000ms = 10.0s
45 # M=20 calls<= 20 cost<= $0.40 lat<= 20000ms = 20.0s

```

— end of document —